

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

PROYECTO FIN DE CURSO (PFC) — ENTREGA 1

PROYECTO:

TiendaTech: Propuesta y Análisis del Sistema Distribuido

MATERIA:

Aplicaciones Distribuidas

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERON

CURSO / PERÍODO ACADÉMICO:

7mo A — Período Académico 2026

INTEGRANTES Y ROLES:

Arquitecto: Jhinson Stalyn Aucatoma Celorio

Líder de Desarrollo: Jeremy Ruperto Gaibor Rodriguez

Responsable de Calidad: Andy Paul Sanchez Pilaloa

Responsable de Documentación: José Alejandro Lozano Morales

FECHA DE ENTREGA:

27 de mayo de 2026

Resumen Ejecutivo

El proyecto **TiendaTech** consiste en el diseño e implementación de una plataforma nacional de comercio electrónico centralizada y especializada en la venta de componentes de hardware y ensamblado personalizado de computadoras en Ecuador. El sistema resuelve una problemática crítica en el mercado tecnológico local: la asimetría de información y el alto riesgo de incompatibilidad física o lógica al que se enfrentan los usuarios —desde entusiastas hasta profesionales— cuando intentan estructurar configuraciones de hardware personalizadas de forma independiente. Para mitigar esto, la plataforma integra una herramienta avanzada de ensamblado automatizado asistida por un agente inteligente conversacional.

El ecosistema digital está dirigido a dos perfiles principales de usuarios a nivel de todo el país: compradores finales que buscan adquirir componentes individuales garantizados , y usuarios técnicos o armadores que demandan un entorno asistido para validar la interoperabilidad de componentes críticos como procesadores, tarjetas madres, memorias RAM y fuentes de poder antes de efectuar una inversión monetaria.

La adopción de una arquitectura de sistema distribuido es la solución tecnológica apropiada debido a la naturaleza heterogénea y de alta concurrencia de los módulos requeridos. La validación de compatibilidad en tiempo real , el cálculo síncrono de costos e impuestos , el procesamiento seguro de pasarelas de pago múltiples (PayPal, tarjetas, transferencias) y la generación inmediata de facturación electrónica indexada, demandan un aislamiento de procesos para garantizar una alta disponibilidad, tolerancia a fallos y escalabilidad elástica. Delegar la carga computacional de la lógica de compatibilidad y el procesamiento del lenguaje natural del asistente inteligente en microservicios independientes evita que los picos de tráfico en el catálogo saturen las transacciones críticas de pago.

El alcance del presente período académico se delimita estrictamente al desarrollo de la primera fase evolutiva del sistema distribuido. Esto comprende la especificación formal de los requerimientos, el modelado arquitectónico de los servicios web, el despliegue de las bases de datos distribuidas y la construcción del núcleo lógico de los módulos de autenticación, carrito de compras, motor de compatibilidad de hardware y la interfaz base del asistente inteligente.

1. Introducción

1.1. Contexto y Problema Cuantificado

En la última década, el comercio electrónico en el Ecuador ha experimentado un crecimiento exponencial. Según datos oficiales de la Cámara Ecuatoriana de Comercio Electrónico (CECE), las transacciones digitales en el país han alcanzado una facturación anual que supera los 4,000 millones de dólares, donde la categoría de bienes tecnológicos y hardware informático se posiciona de forma consistente entre los tres sectores con mayor volumen de transacciones y demanda de mercado. Dentro de este ecosistema macroeconómico digital, el aprovisionamiento de computación personalizada de alto rendimiento exige plataformas comerciales que superen las capacidades de los catálogos estáticos tradicionales.

El despliegue de plataformas nacionales complejas orientadas al comercio especializado requiere de la evolución de las arquitecturas de software tradicionales hacia sistemas distribuidos capaces de procesar información heterogénea en tiempo real [1]. En este escenario nace **TiendaTech**, una plataforma centralizada de cobertura nacional diseñada no solo para capturar una cuota del flujo comercial cuantificado del hardware en el país, sino para automatizar la asistencia técnica mediante sistemas de software interactivos.

1.2. Definición del Problema y Brecha de Requerimientos Formales

El proceso de selección e integración de componentes para el armado de un computador representa un desafío crítico debido a la alta asimetría de información y a las estrictas dependencias de compatibilidad física, eléctrica y lógica entre las piezas. Un usuario se enfrenta al riesgo constante de adquirir componentes que presentan colisiones tecnológicas insalvables (como la disparidad de zócalos entre procesador y placa madre). En el contexto del e-commerce ecuatoriano actual, la mayoría de los sistemas operan bajo esquemas monolíticos que aíslan el inventario de las especificaciones de ingeniería del producto.

Aquí se identifica una evidente **brecha de requerimientos formales (Brecha RQ)**: las plataformas comerciales vigentes cumplen con requerimientos transaccionales básicos (gestión de carrito, pasarela de pagos), pero carecen de requerimientos lógicos y formales de validación cruzada en tiempo real. Al no existir especificaciones formales automatizadas que validen las restricciones tecnológicas de los productos antes de la compra, el error del consumidor se detecta únicamente tras la entrega física, lo que desencadena ciclos ineficientes de devoluciones y pérdidas económicas para la empresa, evidenciando un vacío técnico que los sistemas monolíticos convencionales no pueden resolver.

1.3. Justificación de la Solución y Contribución Delimitada

Para mitigar la brecha identificada, **TiendaTech** se concibe bajo el paradigma de un sistema distribuido fundamentado en una arquitectura de microservicios elásticos. La naturaleza síncrona de las validaciones de hardware, el cálculo de impuestos y la facturación electrónica nacional exigen un aislamiento de procesos para garantizar alta disponibilidad y

tolerancia a fallos [2]. El motor de compatibilidad se separa en un microservicio especializado que soluciona los conflictos de hardware mediante modelos lógicos de programación por restricciones [3], interpretando las reglas ante el usuario mediante enfoques de co-clustering explicable [4]. Asimismo, la brecha de soporte se cubre con un microservicio de Asistente Inteligente basado en Modelos de Lenguaje Grandes (LLMs) y arquitecturas RAG [5].

La **contribución delimitada** del presente proyecto durante este ciclo académico se restringe formalmente al diseño arquitectónico, modelado y despliegue del ecosistema distribuido centralizado para TiendaTech. Esto comprende la especificación del API Gateway, la construcción de las bases de datos distribuidas y la implementación del núcleo lógico de los módulos de autenticación, carrito de compras, motor de compatibilidad y la interfaz cognitiva base del asistente. Con esta delimitación, el proyecto no pretende resolver la cadena de suministro física, sino entregar una solución de software de alta disponibilidad que resuelva de raíz la brecha de validación lógica en el comercio electrónico tecnológico del país.

2. Estado del Arte

El desarrollo de plataformas modernas de comercio electrónico optimizadas para la configuración de productos complejos ha impulsado una notable evolución en la literatura científica. A continuación, se presenta una síntesis integradora de los enfoques metodológicos actuales, estructurada mediante el mapeo de problemas y tecnologías de vanguardia indexadas entre los años 2020 y 2026.

3. Modelado Arquitectónico mediante Diagramas C4

Con el objetivo de representar la estructura general del ecosistema distribuido de **TiendaTech**, se emplea el modelo arquitectónico C4, el cual permite describir sistemas de software complejos mediante diferentes niveles de abstracción.

3.1. Diagrama C4-L1: Contexto del Sistema

El diagrama C4 de nivel 1 permite visualizar la interacción global entre los usuarios externos y la plataforma distribuida **TiendaTech**. En este nivel se identifican los actores principales, así como los servicios externos en el flujo general de operación del sistema.

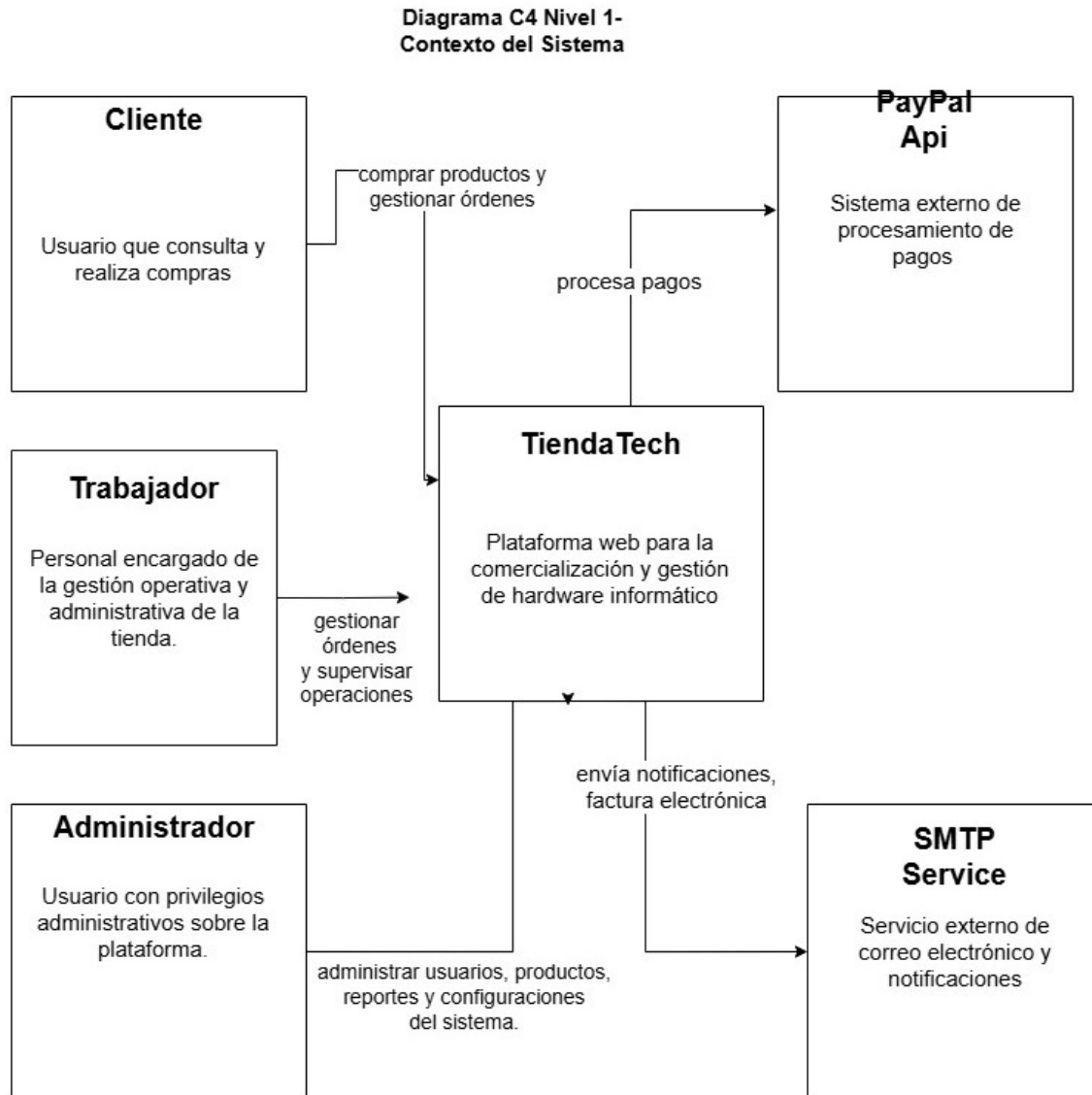


Figura 1: Diagrama C4-L1 del sistema distribuido TiendaTech.

3.2. Diagrama C4-L2: Diagrama de Contenedores

El diagrama C4 de nivel 2 representa la descomposición interna de la arquitectura distribuida de **TiendaTech**.

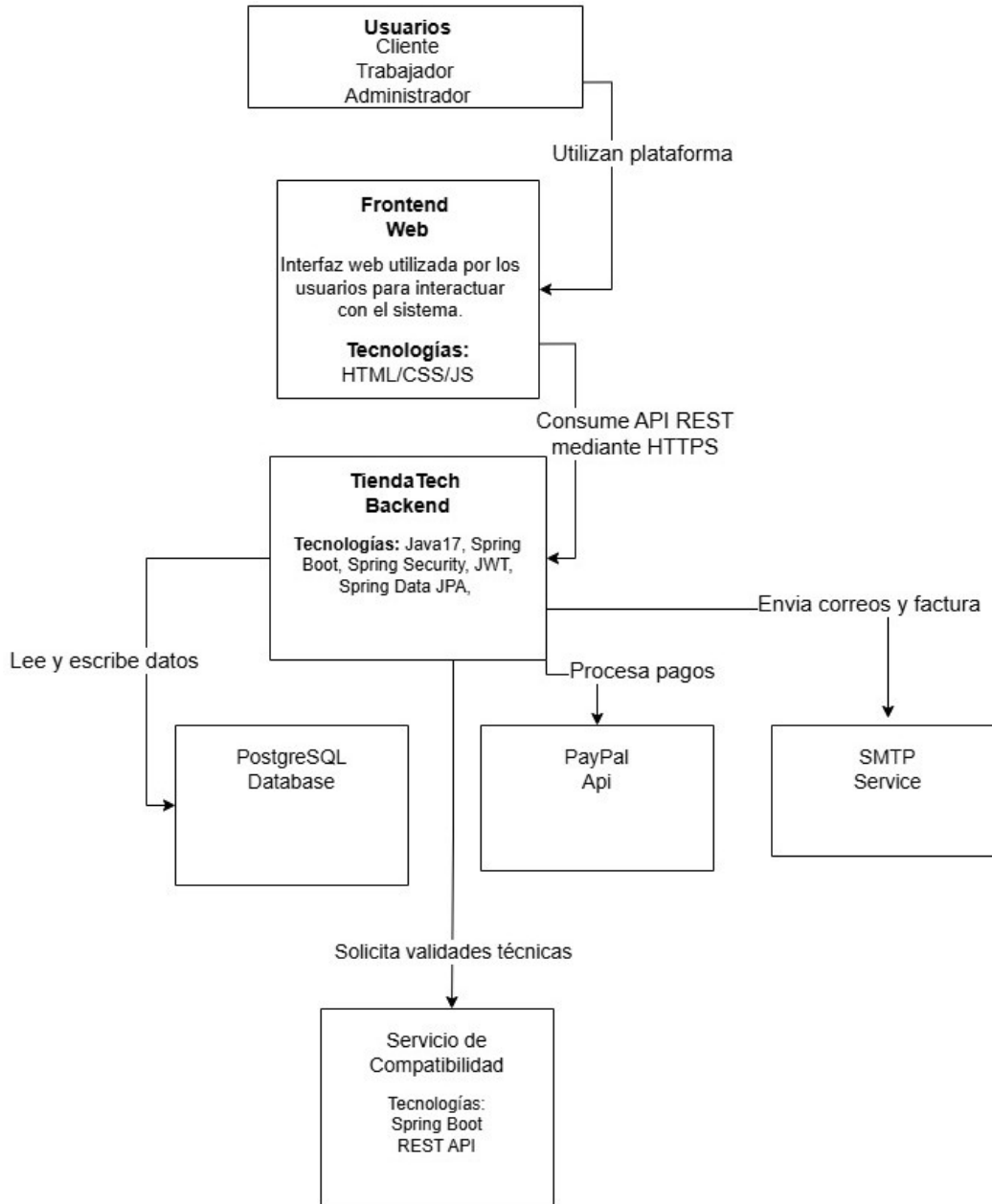


Figura 2: Diagrama C4-L2 de la arquitectura distribuida TiendaTech.

3.3. Mapeo de Problemas y Enfoques Metodológicos

El primer núcleo problemático identificado en la literatura corresponde a la rigidez y la falta de contextualización en las recomendaciones tradicionales de catálogo. Trabajos como el de Vinayababu y Raju [1] y Khadka y Sthapit [6] abordan la transición desde el filtrado colaborativo clásico hacia arquitecturas híbridas distribuidas, demostrando que la diversificación y la escalabilidad del catálogo son críticas para mitigar la sobrecarga de información en entornos transaccionales de alta concurrencia. En este sentido, la optimización matemática

de estos catálogos ha sido evaluada empíricamente por Tokala et al. [7], quien demuestra que variantes avanzadas de factorización de matrices (como SVD++) minimizan el error cuadrático medio (RMSE) en el backend analítico de las plataformas comerciales.

El segundo desafío crítico radica en la resolución de colisiones lógicas durante la personalización adaptativa de productos. El enfoque metodológico de Velten y Hamkins [3] introduce la programación por restricciones (Constraint Programming) como una solución óptima para modelar tablas de restricciones de ingeniería informando en tiempo real al usuario sobre conflictos mínimos y sus respectivas resoluciones. Para complementar este dinamismo sin degradar la experiencia de usuario, Ben Ncir y Alattas [4] proponen modelos de *co-clustering* explicable que no solo agrupan las preferencias multidimensionales de los clientes, sino que dotan al sistema de interpretabilidad, explicando de forma transparente la razón técnica de una sugerencia o restricción.

Finalmente, la brecha de interacción interactiva se ha visto revolucionada por el procesamiento de lenguaje natural. Chihab et al. [8] demuestran la viabilidad de clasificar automáticamente perfiles de usuarios a partir de texto no estructurado para dirigir recomendaciones basadas en competencias o necesidades explícitas. Asimismo, Munson et al. [5] y Sabiri et al. [2] exponen la cúspide actual de esta evolución: la hibridación de Modelos de Lenguaje Grandes (LLMs) con sistemas de recomendación tradicionales, transitando de estructuras rígidas basadas en matrices hacia sistemas cognitivos capaces de mantener diálogos fluidos utilizando arquitecturas Transformer y recuperación aumentada de datos (RAG).

3.4. Tabla Comparativa de Trabajos Relacionados

Para sintetizar los enfoques metodológicos y contrastar el estado actual de la tecnología, la Tabla 1 condensa las contribuciones, ventajas y limitaciones de las fuentes indexadas seleccionadas.

Cuadro 1: Matriz comparativa del Estado del Arte.

Referencia Indexada	Enfoque Metodológico / Tecnología	Ventajas Principales	Limitaciones Detectadas
Vinayababu y Raju [1]	Encuesta de Sistemas Híbridos	Mapeo global de arquitecturas modernas.	No profundiza en validaciones lógicas de ingeniería.
Sabiri et al. [2]	Sistemas basados en Calidad y Microservicios	Alta disponibilidad y métricas de rendimiento.	Complejidad elevada en el acoplamiento de datos.
Chihab et al. [8]	NLP y Clasificación Supervisada	Clasificación precisa de perfiles (96.38 %).	Requiere datos pre-etiquetados masivos.
Ben Ncir y Alattas [4]	Overlapping Co-Clustering (OCCMM)	Alta interpretabilidad y explicabilidad del sistema.	Elevado consumo computacional en matrices densas.
Tokala et al. [7]	Factorización de Matrices (SVD/SVD++)	Optimización matemática predictiva del catálogo.	Es un proceso estático, no apto para flujos síncronos.
Velten y Hamkins [3]	Programación por Restricciones (CP-Solver)	Resolución de conflictos mínimos en tiempo real.	Limitado a configuraciones industriales rígidas.
Munson et al. [5]	Modelos de Lenguaje Grande (LLMs)	Interacción conversacional altamente contextual.	Riesgo latente de alucinación de datos técnicos.

3.5. Declaración del Vacío Científico (Gap Statement)

A pesar de los significativos avances evidenciados en el mapeo previo, la revisión de la literatura científica indexada expone un vacío técnico crítico (*Gap Statement*): **existe una desconexión metodológica entre los entornos de asistencia conversacional inteligente basados en LLMs y los motores rígidos de validación de ingeniería por restricciones lógicas operando en arquitecturas distribuidas de alta disponibilidad.** Mientras que las investigaciones actuales solucionan el procesamiento conversacional de forma aislada [5], o resuelven las colisiones de componentes de manera puramente matemática y confinada a sistemas industriales cerrados

[3], no se evidencia un marco arquitectónico integrado en el e-commerce de consumo masivo que unifique ambos mundos bajo microservicios elásticos. La mayoría de los desarrollos conversacionales carecen de un motor de restricciones síncrono que impida la alucinación de compatibilidades de hardware informático, mientras que los solvers de restricciones tradicionales carecen de interfaces cognitivas en lenguaje natural accesibles para el usuario común. El proyecto **TiendaTech** se propone mitigar este vacío científico, contribuyendo con el diseño y despliegue de un ecosistema distribuido donde un API Gateway orquesta concurrentemente un asistente conversacional RAG y un motor de restricciones elástico de compatibilidad de hardware a escala nacional.

3.6. Registros de decisiones arquitectónicas ADR

3.6.1. ADR-01: Arquitectura basada en microservicios

Estado: Aceptado.

Alternativas consideradas: Arquitectura monolítica, descartada por el alto acoplamiento entre módulos; y arquitectura en capas N-tier, descartada por sus limitaciones para escalar servicios de forma independiente.

Decisión: Se adopta una arquitectura basada en microservicios, donde los módulos principales del sistema funcionan como servicios independientes: autenticación, catálogo, carrito, pagos, facturación, motor de compatibilidad y asistente inteligente.

Justificación: Esta arquitectura permite separar responsabilidades, escalar módulos específicos y reducir el impacto de fallos. Por ejemplo, un problema en el catálogo no debería afectar directamente el procesamiento de pagos.

Consecuencia: Mejora la escalabilidad, disponibilidad y mantenibilidad del sistema, aunque incrementa la complejidad de comunicación entre servicios.

3.6.2. ADR-02: Uso de API Gateway

Estado: Aceptado.

Alternativas consideradas: Comunicación directa entre cliente y microservicios, descartada por riesgos de seguridad y mayor acoplamiento; y Backend for Frontend, descartado porque en esta fase solo se contempla un cliente web principal.

Decisión: Se implementa un API Gateway como punto de entrada único para las solicitudes del cliente web.

Justificación: El API Gateway centraliza el enrutamiento, la autenticación y el control de acceso hacia los microservicios. Esto evita que el cliente se comunique directamente con cada servicio y mejora la seguridad del sistema.

Consecuencia: Facilita el control de solicitudes y la organización de la arquitectura, pero se convierte en un componente crítico que debe manejarse con tolerancia a fallos.

3.6.3. ADR-03: Motor de compatibilidad como microservicio independiente

Estado: Aceptado.

Alternativas consideradas: Incluir la lógica de compatibilidad dentro del catálogo, descartado por mezclar responsabilidades; y validar la compatibilidad en el frontend, descartado por riesgos de manipulación y baja confiabilidad.

Decisión: Se implementa el motor de compatibilidad de hardware como un microservicio especializado.

Justificación: La validación entre procesador, placa madre, RAM, GPU, almacenamiento y fuente de poder requiere reglas técnicas que pueden cambiar con frecuencia. Separar este módulo permite actualizar las reglas sin modificar el catálogo, el carrito o los pagos.

Consecuencia: Mejora la precisión y mantenibilidad del sistema, aunque requiere comunicación constante con el catálogo de productos para mantener actualizada la información técnica.

3.7. Métricas de calidad basadas en ISO/IEC 25010

Las métricas se definen para evaluar atributos de calidad relevantes en **TiendaTech**, especialmente rendimiento, disponibilidad, fiabilidad, seguridad, mantenibilidad y compatibilidad funcional.

Cuadro 2: Métricas de calidad basadas en ISO/IEC 25010 para TiendaTech.

Característica	Métrica conocida	Forma de medición / fórmula	Umbral objetivo
Rendimiento	Tiempo de respuesta P95	Ordenar los tiempos de respuesta y tomar el valor donde se ubica el 95 % de solicitudes.	≤ 500 ms
Rendimiento	Throughput	$\frac{\text{Total de solicitudes procesadas}}{\text{Tiempo total de prueba}} \times 100$	≥ 30 req/s
Disponibilidad	Uptime	$\frac{\text{Tiempo total de operación}}{\text{Tiempo total de prueba}} \times 100$	$\geq 99\%$
Fiabilidad	Error Rate	$\frac{\text{Solicitudes fallidas}}{\text{Total de solicitudes}} \times 100$	$\leq 1\%$
Mantenibilidad	MTTR	$\frac{\text{Número de incidentes corregidos}}{\text{Tiempo total de reparación}} \times 100$	≤ 2 horas
Seguridad	Tasa de bloqueo de accesos no autorizados	$\frac{\text{Intentos no autorizados bloqueados}}{\text{Total de intentos no autorizados}} \times 100$	$\geq 99\%$
Compatibilidad funcional	Tasa de pruebas funcionales aprobadas	$\frac{\text{Casos de prueba aprobados}}{\text{Total de casos de prueba ejecutados}} \times 100$	$\geq 95\%$

Estas métricas cubren los aspectos más importantes de **TiendaTech**. En esta fase se priorizan el rendimiento, la disponibilidad, la fiabilidad, la seguridad y la compatibilidad funcional, porque afectan directamente la experiencia del usuario y la correcta validación de los componentes de hardware.